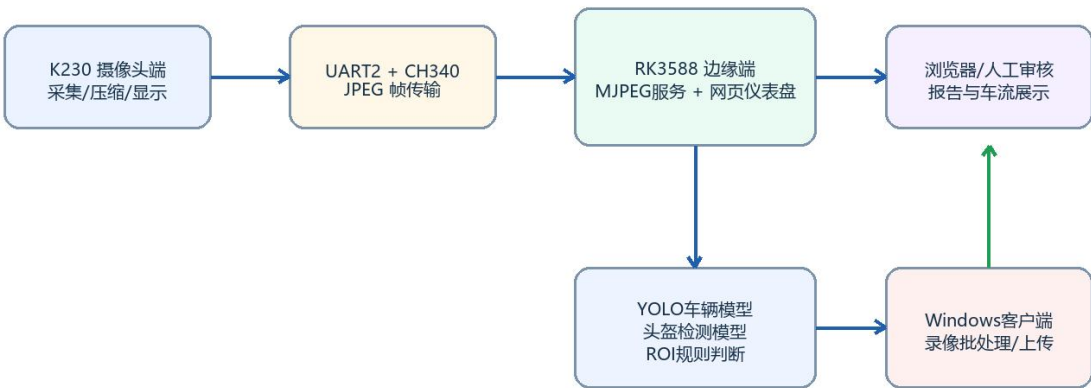


基于 K230 与 RK3588 的 智能交通违法识别与审核系统

作品定位：本作品面向路口交通秩序辅助管理场景，完成从摄像头采集、边缘端检测、网页端审核到报告输出的闭环设计。

系统总体框图



数据链路：K230 实时帧 -> RK3588 解码与检测 -> 网页审核/报告；Windows 可离线处理录像并上传违规事件和车流量。

图 1 系统总体框图

作品名称

基于 K230 与 RK3588 的智能交通违法识别与审核系统

摘要

本作品设计并实现了一套面向道路交通场景的智能违法识别与审核系统。系统以 K230 作为前端摄像头采集与显示节点，通过 UART2 串口和 CH340 将压缩后的 JPEG 帧传输至 RK3588；RK3588 侧运行视频接收服务和网页仪表盘，提供实时画面、AI 叠加画面、ROI 标定、人工审核、去重确认、车流量统计和报告生成等功能。同时，系统提供 Windows 客户端，支持对离线录像进行批处理分析，并将违规截图、置信度、违法原因、车流状态等信息上传至网页端统一审核。算法层面采用基础车辆检测模型识别机动车、摩托车等目标，结合红绿灯 ROI 色域判断、斑马线多边形越线判断、目标跟踪 ID 继承和头盔检测模型，实现闯红灯与未戴头盔事件的抓拍。系统支持快速结构化报告与 AI 润色报告两种输出方式，兼顾现场演示的实时性和正式留档的可读性。测试结果表明，该方案能够在边缘计算条件下完成图像采集、实时展示、辅助识别、人工复核与报告输出闭环，为校园道路、园区出入口和小型路口的交通安全管理提供了一种低成本、可扩展的实现思路。

第一部分 作品概述

1.1 功能与特性

本作品围绕“实时画面接入、交通目标识别、违法事件抓拍、人工审核确认、报告自动生成”五个环节展开设计。前端 K230 负责采集画面并进行串口推流，RK3588 负责网页服务、模型推理和数据汇总，Windows 客户端负责离线录像批处理。系统能够在网页端显示 K230 实时画面与 AI 叠加结果，支持人工框选红绿灯区域和斑马线区域，并将闯红灯、未戴头盔等疑似事件以截图形式进入人工审核队列。

- 实时视频链路：K230 摄像头采集后压缩为 JPEG，通过 UART2 + CH340 传输至 RK3588。
- 双模型检测：基础模型负责车辆类别检测，头盔模型仅针对摩托车/电动车相关目标进行辅助判断。
- 规则融合：红灯色域判断、斑马线 ROI 越线判断、目标跟踪和稳定帧策略共同降低误判。
- 审核闭环：网页端提供确认、驳回、重复标记、清空记录、快速报告和 AI 润色报告。
- 车流统计：系统按当前检测车辆数量显示畅通、缓行、拥堵状态，并支持客户端上传车流量。

部署链路与服务端口



说明：5001 负责原始视频流，5000 负责网页显示、ROI标定、AI叠加画面、人工审核和报告生成。

图 2 K230 与 RK3588 部署链路

1.2 应用领域

系统适用于校园道路、园区出入口、实验教学平台、小型路口交通秩序辅助管理等场景。与传统固定式交通执法设备相比，本作品更强调低成本、可复现和可二次开发：硬件端使用开发板与常见 USB 转串口模块，软件端采用 Python、OpenCV、Flask 和 YOLO 模型，便于在教学、竞赛和原型验证中快速部署。

1.3 主要技术特点

- 端边协同：K230 负责采集与显示，RK3588 承担网页服务和模型推理，Windows 端承担录像批处理。
- 多源互通：实时流检测结果和桌面端录像分析结果统一进入同一网页审核数据库。
- 可视化标定：网页端提供红绿灯矩形 ROI 和斑马线多边形 ROI 标定，降低部署门槛。
- 稳定性优化：头盔状态采用间隔检测和稳定判断，目标跟踪支持短时间 ID 继承。
- 可扩展接口：事件上传、车流上传、报告生成、图片访问均封装为 HTTP API。

1.4 主要性能指标

表 1 系统主要性能与参数

指标项	当前实现/设计值	说明
视频输入	K230 实时串口流、Windows 本地视频	支持实时与离线两类来源
实时网页端口	5000	仪表盘、审核、报告、ROI 标定
MJPEG 视频端口	5001	RK3588 对外提供 /video
模型输入尺寸	约 256-320 可配置	根据 RK3588 性能在速度和精度间权衡

车流状态阈值	0-4 畅通，5-9 缓行，10+拥堵	可通过环境变量调整
事件信息	时间、截图、原因、置信度、来源、元数据	用于人工复核和报告生成

1.5 主要创新点

- 提出“实时端 + 离线端 + 网页审核端”的双端互通结构，将实时检测和录像批处理结果统一管理。
- 在轻量硬件条件下实现红绿灯 ROI、斑马线 ROI、车辆检测、头盔检测和车流统计的融合判断。
- 将 AI 自动识别结果设计为“待审核事件”，保留人工确认环节，提升系统在复杂场景中的可用性。
- 报告生成采用快速结构化模式与 AI 润色模式并存，兼顾稳定性和展示效果。

1.6 设计流程

核心算法流程

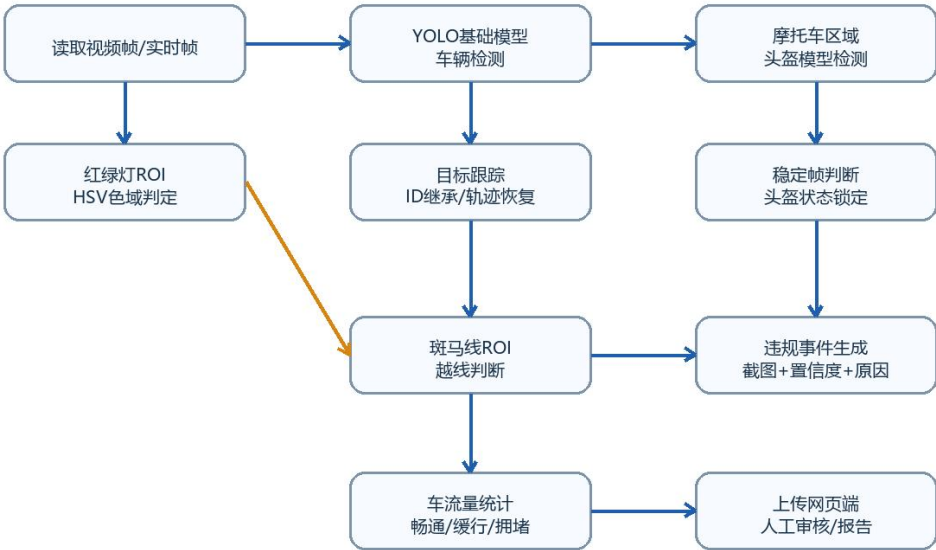


图 3 识别与审核核心流程

1. 完成 K230 摄像头、显示屏和 UART2 引脚配置，解决传感器识别、画面翻转和 JPEG 帧发送问题。
2. 在 RK3588 端实现串口接收和 MJPEG 服务，保证浏览器可稳定查看 K230 实时画面。
3. 搭建网页仪表盘，实现直播显示、AI 叠加、ROI 标定、事件审核和报告生成。
4. 开发 Windows 客户端，实现录像导入、双模型检测、违规截图和结果上传。
5. 围绕卡顿、头盔误检、红灯阈值、报告超时、人工审核失效等问题持续优化。

第二部分 系统组成及功能说明

2.1 整体介绍

系统整体由 K230 摄像头端、RK3588 边缘网页端、Windows 客户端和模型/数据层四部分组成。K230 端采集画面后以固定格式发送 JPEG 帧；RK3588 端解析帧流并对外提供 MJPEG 视频，同时加载车辆模型和头盔模型进行在线检测；Windows 客户端可处理本地录像并将结果上传；网页端则作为统一交互入口，承担标定、审核、统计和报告输出。

软件模块组成



图 4 软件模块组成

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件系统主要包括 K230 开发板、摄像头模组、800×480 DSI Display V2.0 显示屏、CH340 USB 转串口模块、RK3588 开发板以及供电与连接线。K230 负责图像采集和本地显示，CH340 负责将 K230 的 UART2 串口数据转换为 RK3588 可识别的 USB 串口设备，RK3588 负责接收数据、运行网页服务和模型推理。

表 2 硬件模块说明

硬件模块	作用	关键连接/参数
K230 开发板	摄像头采集、本地显示、JPEG 压缩和 UART 发送	UART2 TX/RX、DSI 显示屏、摄像头接口
摄像头模组	采集路口画面	当前代码支持自动尝试 sensor id 0/1/2
800×480 DSI 显示屏	K230 本地预览画面	Display.ST7701, 800×480

CH340 模块	串口转 USB	连接 K230 UART2 与 RK3588 USB
RK3588 开发板	视频服务、网页端、YOLO 推理	/dev/ttyUSB0、端口 5000/5001

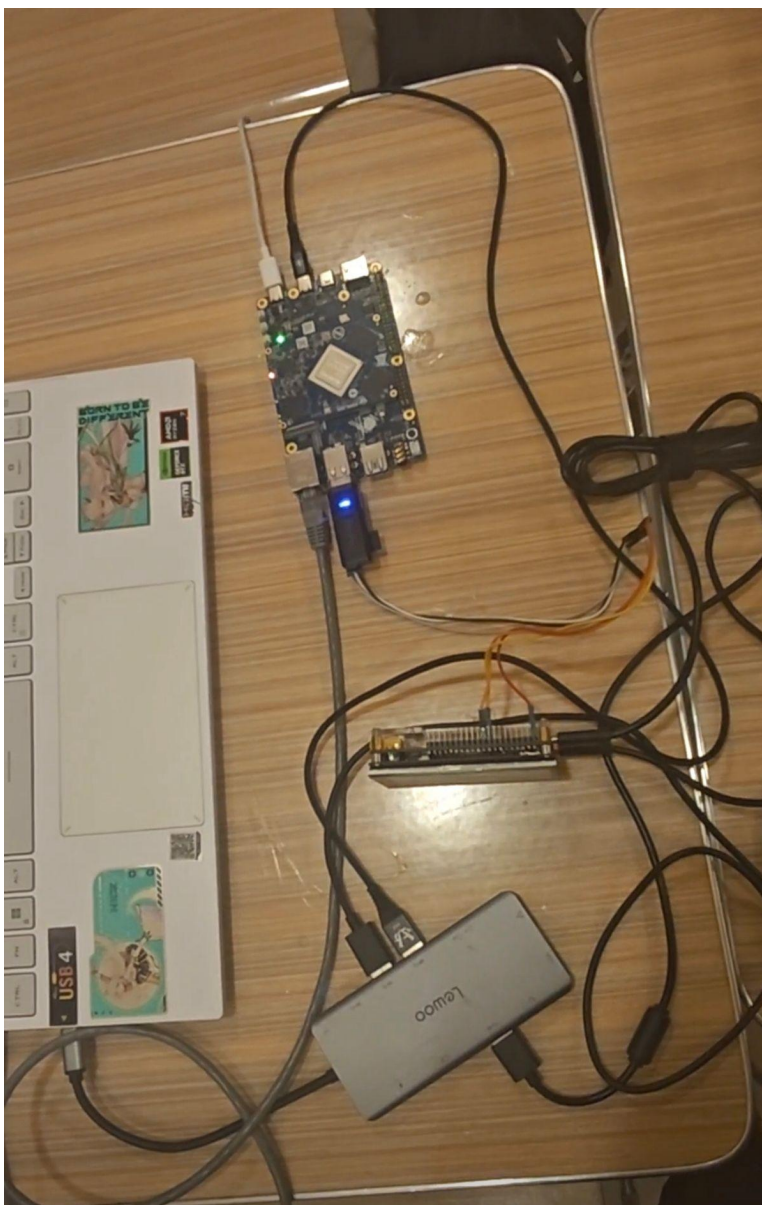


图 5 整机全局实物照片占位

2.2.2 机械设计介绍

本作品以功能验证为主，机械结构采用开放式组合安装方式。摄像头需要固定在能够覆盖红绿灯、斑马线和车辆通行区域的位置，K230 与显示屏可安装在同一支架或面板上，RK3588 可放置在旁侧作为边缘计算节点。实际应用时可进一步设计外壳、散热孔、固定支架和防误触接口。

2.2.3 电路各模块介绍

电路连接的核心是串口链路。K230 端通过 FPIOA 将 GPIO5 配置为 UART2_TXD，将 GPIO6 配置为 UART2_RXD；CH340 模块负责电平与 USB 接口转换；RK3588 端识别为 /dev/ttyUSB0 后由 Python 串口程序读取帧长度和帧数据。为提升画面流畅度，系统将串口波特率调高至 1.5 Mbps 左右，并对帧大小设置上限以避免异常数据阻塞。

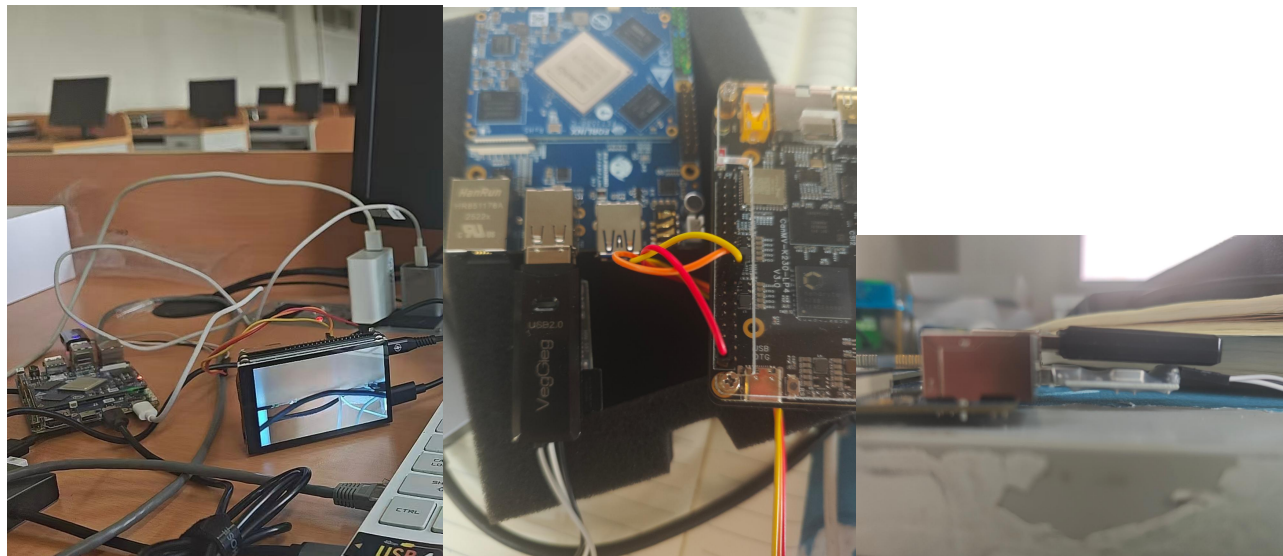
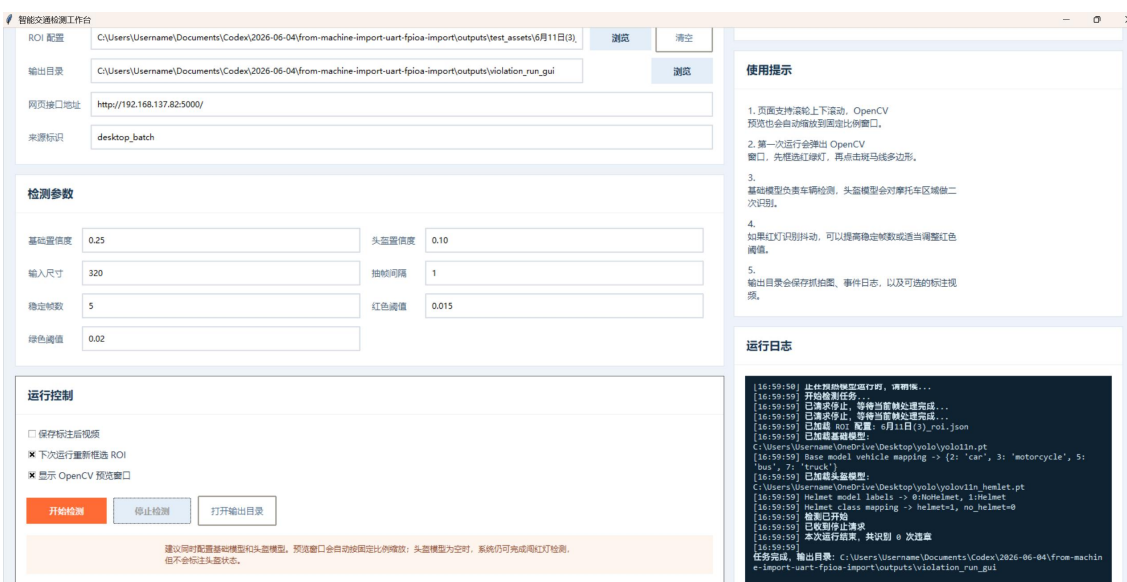
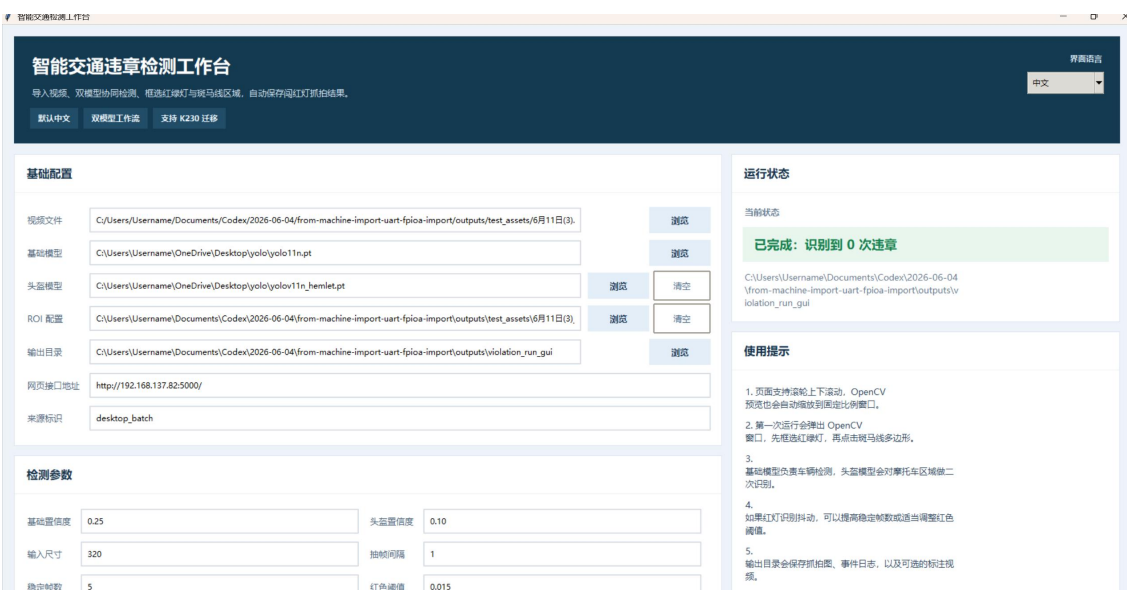
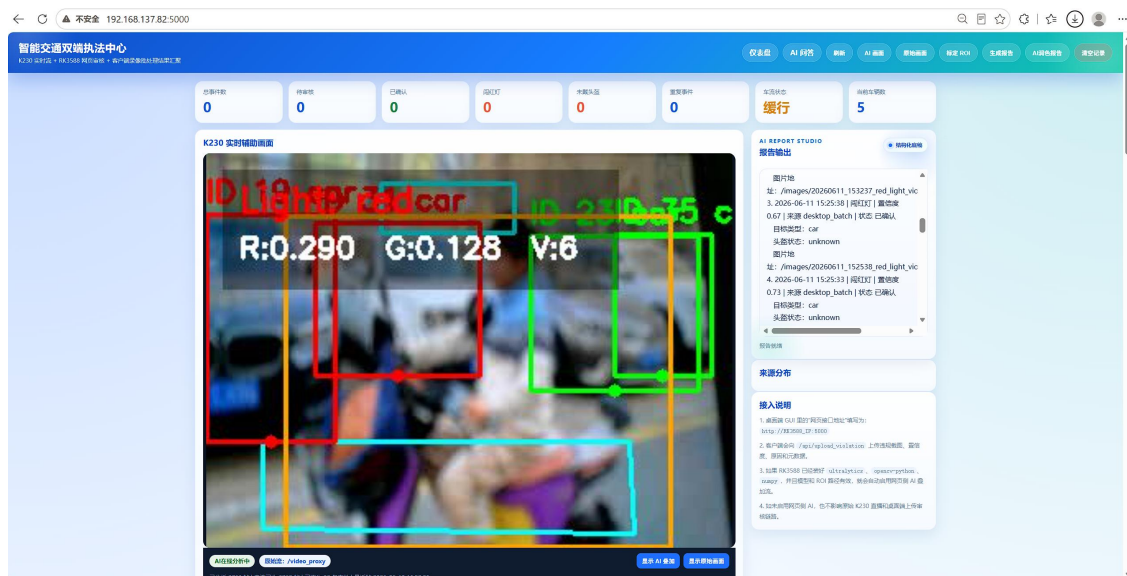


图 6 硬件连接与电路照片占位

2.3 软件系统介绍

2.3.1 软件整体介绍

软件系统分为采集层、传输层、推理层、交互层和报告层。采集层由 K230 MicroPython 程序实现；传输层由串口协议和 MJPEG 服务实现；推理层由 YOLO 车辆检测、头盔检测、红绿灯色域判断和斑马线越线判断组成；交互层为 Flask 网页仪表盘和 Windows Tkinter 客户端；报告层提供结构化统计和 AI 润色输出。



智能交通违法识别与审核系统

图 7 网页端功能与客户端功能布局示意

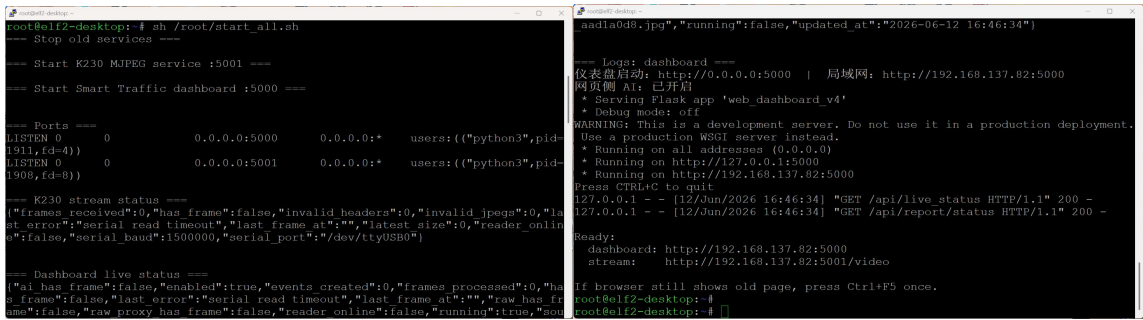


图 8 一键启动脚本

2.3.2 软件各模块介绍

表 3 软件模块输入输出

模块	主要文件	输入	输出
K230 推流	k230_uart_stream_fast.py	摄像头帧	JPEG 长度头 + JPEG 数据
RK 视频服务	mjpeg_server_fast.py	/dev/ttyUSB0 串口帧	/video MJPEG 流、 /status 状态
网页仪表盘	web_dashboard_v4.py	MJPEG 流、模型、ROI、 上传事件	审核页面、AI 叠加 流、报告
Windows 客户端	red_light_violation_gui.py	视频、模型、ROI 配置	检测结果、截图、上 传事件
核心算法	red_light_violation_prototype.py	视频帧、模型输出	违规抓拍、车流量、 元数据

关键判断逻辑包括：当红灯状态成立且车辆底部中心点由斑马线外进入斑马线多边形区域时，系统生成闯红灯疑似事件；当目标类型为摩托车且头盔模型在稳定周期内判定为 no_helmet 时，系统生成未戴头盔疑似事件；车流量统计则基于当前未丢失的跟踪目标数量，并按阈值映射为畅通、缓行或拥堵。

第三部分 完成情况及性能参数

3.1 整体介绍

目前系统已经形成可演示的端到端闭环：K230 能采集并发送实时画面，RK3588 能显示直播并运行网页端 AI，Windows 客户端能够导入录像并运行双模型检测，网页端能够接收实时端和客户端上传的事件，人工审核后可生成报告。

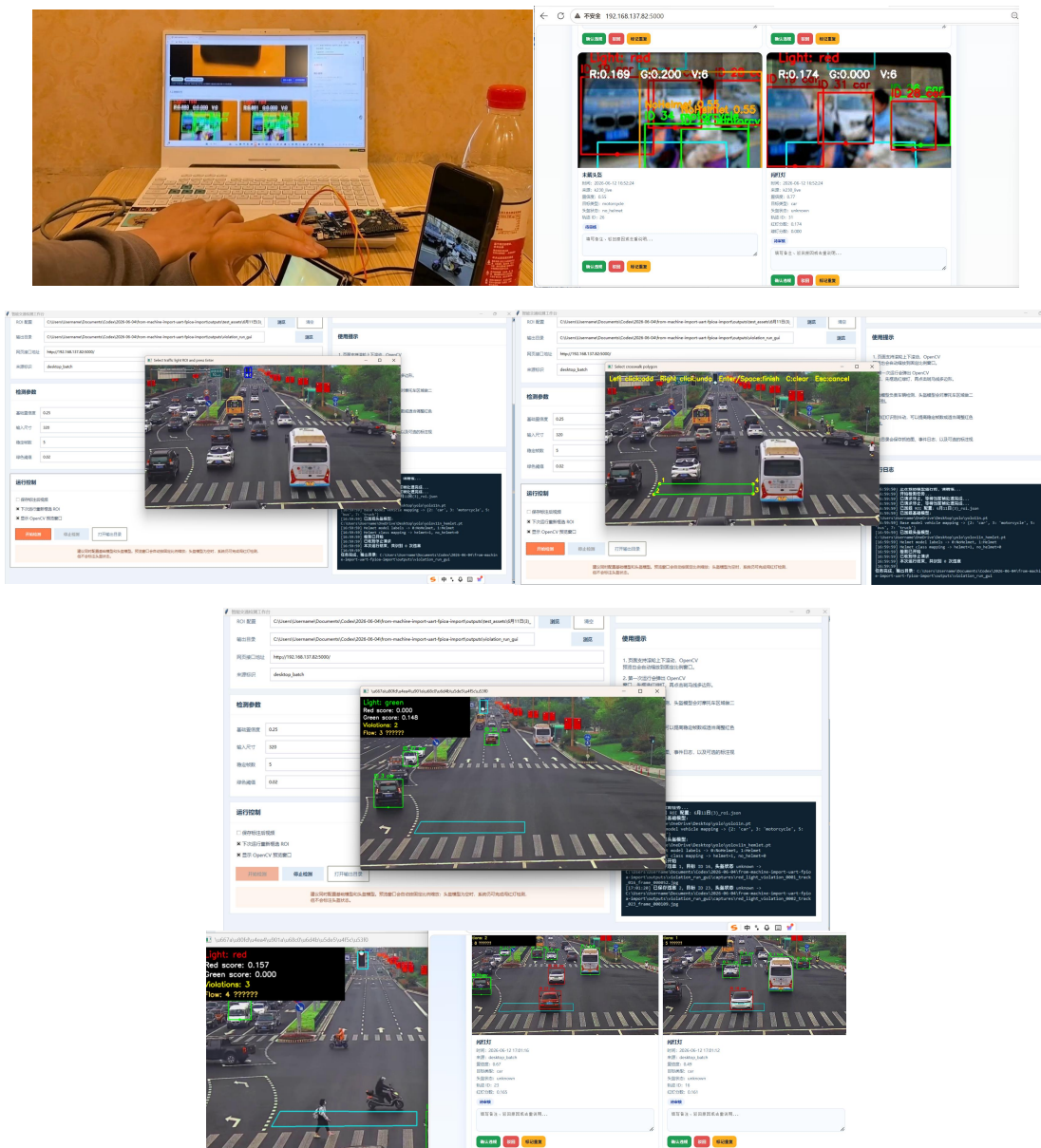


图 9 现场演示与软件界面照片占位

3.2 工程成果

3.2.1 机械成果

完成 K230、摄像头、显示屏、CH340 和 RK3588 的联调验证。当前阶段以开放式硬件连接为主，后续可进一步设计固定支架和外壳结构，以提升现场部署稳定性。

3.2.2 电路成果

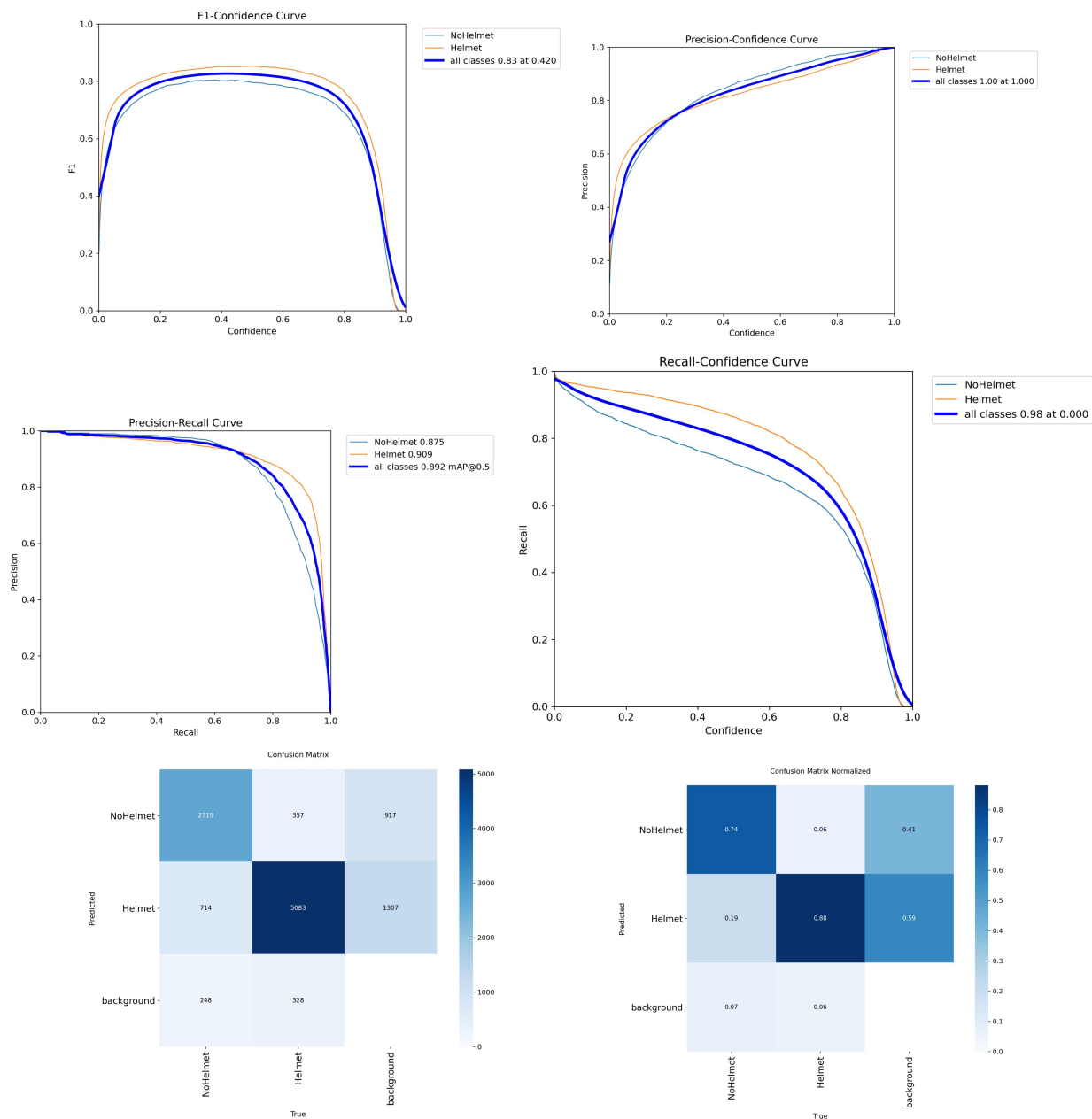
完成 K230 UART2 与 CH340 的串口通信配置，RK3588 能通过 /dev/ttyUSB0 接收帧数据。串口帧采用 4 字节大端长度头 + JPEG 数据的形式，便于接收端进行同步、校验和异常恢复。

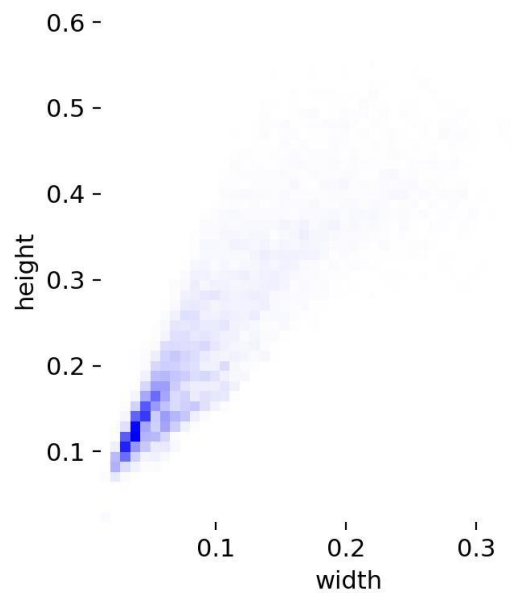
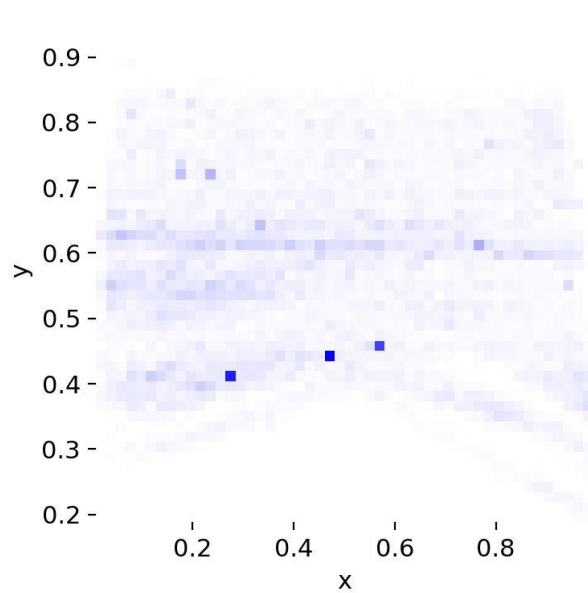
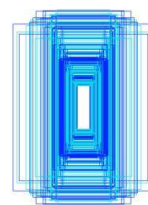
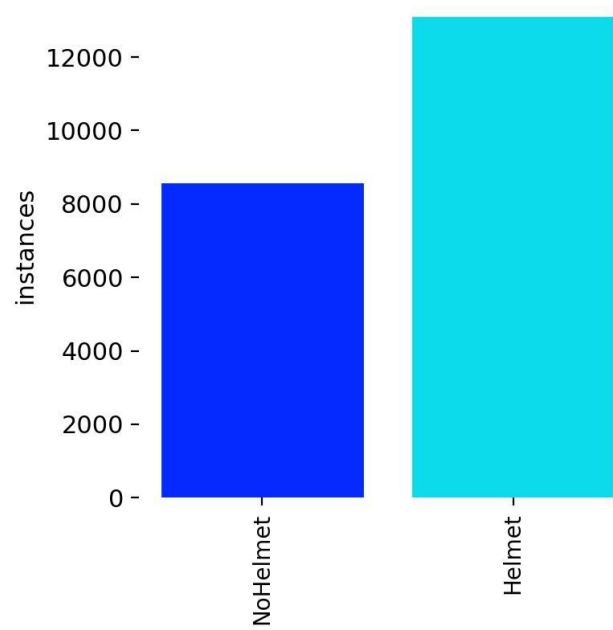
3.2.3 软件成果

- 网页端完成直播显示、AI 叠加、ROI 标定、人工审核、清空记录、报告生成和车流展示。
- Windows 客户端完成视频选择、模型选择、ROI 配置、运行控制、日志输出和结果上传。
- RK3588 端支持 SD 卡 Python 库部署、模型路径配置和一键启动脚本。
- K230 端完成摄像头自动探测、屏幕显示、画面翻转修正和串口高速推流。

3.3 特性成果与性能参数

有关 yolo_helmet.pt 头盔识别模型训练的效果图与参数：





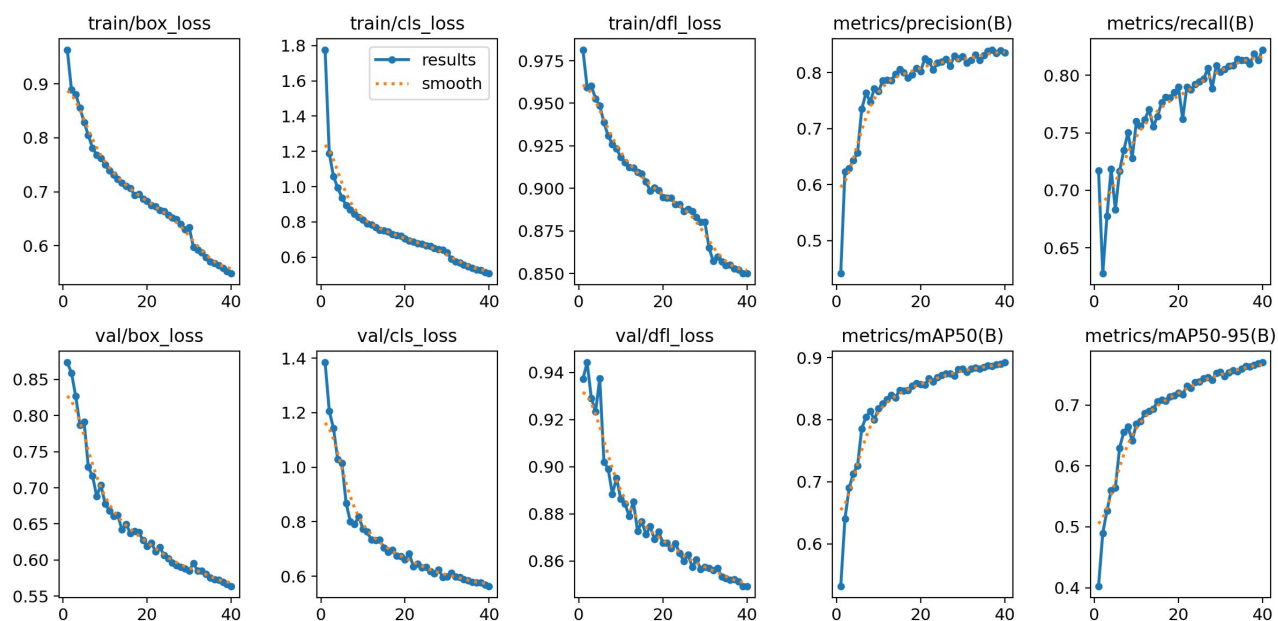


表 4 功能完成情况

功能项	完成情况	备注
实时画面接入	已完成	浏览器可通过网页查看 K230 画面
网页 ROI 标定	已完成	支持红绿灯矩形框和斑马线多边形
闯红灯检测	已完成原型	依赖红灯 ROI、斑马线 ROI 和跟踪轨迹
头盔检测	已完成原型	仅对摩托车相关目标进行辅助判断
人工审核	已完成	确认、驳回、重复标记
报告生成	已完成	快速报告与 AI 润色报告双模式
车流量统计	已完成	网页端与客户端均可发送/显示车流状态

说明：以上性能为当前作品原型阶段的功能完成情况。最终识别准确率、帧率和稳定性会受摄像头视角、光照、串口质量、模型权重和现场车辆类型影响，建议结合实地测试照片或视频进一步补充。

从数据集分布图表 (labels.jpg) 可以看出，本次训练针对安全帽检测任务，包含两个核心目标类别：

1. 样本分布：佩戴头盔 (Helmet) 类样本量约 13000 个，未佩戴头盔 (NoHelmet) 类样本量约 8600 个，Helmet 类样本量为 NoHelmet 类的 1.5 倍，存在一定程度的样本不均衡。
2. 目标特性：从边界框分布来看，所有目标的归一化宽高均较小 (width 集中在 0-0.3 区间，height 集中在 0-0.6 区间)，说明本次数据集以小目标检测场景为主，对模型的小目标特征提取能力有较高要求。

从训练过程曲线（results.png）来看，本次训练共进行 40 个 Epoch，整体训练过程稳定收敛，未出现异常波动：

1. 损失函数变化
2. 训练集与验证集的三项核心损失（边界框损失 box_loss、分类损失 cls_loss、DFL 损失 dfl_loss）均呈现持续下降并最终趋于稳定的趋势。
3. 验证集损失全程未出现上升反弹，说明模型未发生过拟合，具备良好的泛化能力。
4. 评估指标变化
5. 精确率（Precision）、召回率（Recall）、mAP50、mAP50-95 四项核心评估指标，均随训练推进持续上升，在训练后期稳定在较高水平，证明模型在训练过程中持续学习到了有效的目标特征，最终收敛到最优状态。

整体检测精度

从 Precision-Recall（PR）曲线（BoxPR_curve.png）可以看出：

- 模型整体 mAP@0.5（IoU=0.5 时的平均精度）达到 0.892，处于行业较高水平，说明模型整体检测精度优秀，能够有效完成安全帽检测任务。
- 模型在 IoU 从 0.5 到 0.95 的平均精度 mAP50-95 最终达到约 0.78，说明模型不仅能检出目标，还

类别细分性能

两个目标类别的检测性能存在一定差异，与样本量分布高度相关：

1. 佩戴头盔（Helmet）类：AP@0.5 达到 0.909，各项指标均优于另一类别，得益于该类更大的样本量，模型学习更充分。
2. 未佩戴头盔（NoHelmet）类：AP@0.5 达到 0.875，同样具备较高的检测精度，能够满足业务识别需求。

置信度阈值优化

通过不同置信度下的指标曲线，我们可以得到最优的部署阈值：

1. 最优平衡阈值：从 F1-Confidence 曲线（BoxF1_curve.png）来看，模型的最优置信度阈值为 0.42，此时整体 F1 分数（精确率与召回率的调和平均）达到 0.83，实现了两者的最佳平衡。
2. 极端阈值表现：
 1. 当置信度阈值降至 0 时，模型召回率可达 0.98，说明模型几乎能够检出所有目标，仅部分目标的预测置信度较低；
 2. 当置信度阈值提升至 1.0 时，模型精确率可达 1.0，说明高置信度的预测完全不存在假阳性，可靠性极高。

分类错误分析

通过混淆矩阵（confusion_matrix.png、confusion_matrix_normalized.png），我们可以定位模型的主要错误类型：

1. 目标分类错误

1. 未佩戴头盔目标：74% 被正确分类，19% 被误分类为佩戴头盔，7% 被漏检（预测为背景）。该类的核心错误是与佩戴头盔类的混淆，部分特征不明显的未佩戴目标容易被误判。
2. 佩戴头盔目标：88% 被正确分类，仅 6% 被误分类为未佩戴头盔，6% 被漏检，分类准确率更高，错分与漏检率均显著低于另一类别。

2. 假阳性问题

3. 对于背景区域，模型产生的假阳性预测框中，41% 被分类为未佩戴头盔，59% 被分类为佩戴头盔。这说明模型存在一定的误检情况，容易将场景中类似头盔的背景物体（如圆形物体、其他防护装备等）误判为目标。

训练成果总结

本次安全帽检测模型的训练成果优秀：

- 整体 mAP@0.5 达到 0.892，检测精度处于较高水平；
- 训练过程稳定，无过拟合问题，泛化能力良好；
- 能够有效处理小目标检测场景，对两类安全帽目标均具备较高的识别能力，可满足实际业务的部署需求。

后续优化建议

1. 解决样本不均衡：针对未佩戴头盔类样本量较少的问题，可对该类进行针对性数据增强（如翻转、随机裁剪、噪声添加等），提升样本多样性，进一步降低两类之间的错分率。
2. 降低假阳性：增加难负例样本的训练，将容易被误检的背景区域加入训练集，提升模型对背景与目标的区分能力，减少误检情况。
3. 部署阈值调整：
 1. 常规场景下，建议将置信度阈值设置为 0.42，以获得最优的精确率与召回率平衡；
 2. 若业务场景对漏检的容忍度更低（如安全监管场景，不允许漏检未佩戴头盔的人员），可适当降低置信度阈值，优先提升召回率。

第四部分 总结

4.1 可扩展之处

- 模型部署优化：后续可将 YOLO 模型转换为 RKNN 或 K230 支持的 KModel，以提升边缘端推理速度。
- 通信链路优化：若实时性要求更高，可将 UART 串口升级为 USB 摄像头、以太网或 MIPI/CSI 直连方案。
- 数据管理优化：可加入用户权限、事件检索、批量导出、PDF 报告和云端备份功能。
- 算法优化：可加入车牌识别、行人检测、逆行检测、压线检测和多目标轨迹分析。
- 工程外观优化：可设计一体化外壳、固定支架和散热结构，提高现场使用可靠性。

4.2 心得体会

本作品的研发过程体现了嵌入式采集、边缘计算、计算机视觉和网页交互之间的综合协同。最初的难点集中在 K230 摄像头初始化、传感器 ID 识别、屏幕显示、JPEG 数据格式和串口传输稳定性上；在 RK3588 端，主要问题则包括 Python 依赖部署、模型加载、实时画面卡顿、前端状态刷新和报告生成超时。通过逐步拆分链路，可以将“画面是否进入 RK3588”“YOLO 是否启用”“网页是否正确显示”“事件是否进入审核队列”等问题分别定位，最终形成可运行的闭环系统。

本作品的研发过程体现了嵌入式采集、边缘计算、计算机视觉和网页交互之间的综合协同。最初的难点集中在 K230 摄像头初始化、传感器 ID 识别、屏幕显示、JPEG 数据格式和串口传输稳定性上。K230 端的摄像头初始化需要正确配置设备树和 I2C 通信，确保 MIPI 接口的 sensor 驱动与具体模组匹配，过程中经常遇到时钟信号不兼容或寄存器配置遗漏导致无法出图；传感器 ID 识别则要求通过 I2C 读取 chip ID 并与数据手册核对，一旦焊接或排线接触不良就会读取失败；屏幕显示环节需要对齐帧缓冲格式与液晶屏的时序参数，否则会出现撕裂或偏移；JPEG 数据格式的处理上，硬编码器输出的码流需要与解码端的 JPEG 头部定义完全一致，任何 EOI 标记错位都会引起解码失败；串口传输稳定性方面，高波特率下时常出现校验错误和数据粘包，必须精细调节缓冲区大小和流控策略，并加入超时重传机制，才保证了 JPEG 图片在 K230 与 RK3588 之间的可靠搬运。

在 RK3588 端，主要问题则包括 Python 依赖部署、模型加载、实时画面卡顿、前端状态刷新和报告生成超时。部署 Python 依赖时，由于 RK3588 的 Linux 系统中预装的解释器版本与某些视觉库存在 ABI 冲突，需要逐个编译安装特定版本的 NumPy 和 OpenCV，并使用虚拟环境隔离；模型加载阶段，将 YOLO 权重转换为 RKNN 格式后，NPU 内存分配策略和量化精度会直接影响推理首帧延迟，反复调整子图划分才达到了稳定加载；实时画面卡顿源于多路 JPEG 解码与模型推理争抢 CPU 资源，通过把解码管线绑定到独立线程并引入帧跳过逻辑，平衡了吞吐量和延迟；前端状态刷新原先采用短轮询，浏览器页面频繁闪烁且加重服务端负担，改用轻量级消息推送后实现了秒级同步；报告生成超时则是因为在接口内同步等待文字排版与数据统计完成，最后将报告构建解耦为异步任务，状态机回调才彻底消除了阻塞。

通过逐步拆分链路，可以将“画面是否进入 RK3588”“YOLO 是否启用”“网页是否正确显示”“事件是否进入审核队列”等问题分别定位，最终形成可运行的闭环系统。在定位时，画面进入状态可通过查看接收端 JPEG 解码日志和哈希校验确认完整性；YOLO 启停状态依靠进程守护与心跳信号监控；网页显示正确性以 HTTP 状态码和 WebSocket 心跳联合判断；审核队列的积压情况则通过消息队列的消费者偏移量跟踪，这样每段链路都有了明确的分界和诊断手段。

在算法层面，单纯依赖模型检测并不能直接得到可靠的交通违法结论，因此本作品将模型检测结果与红绿灯 ROI、斑马线 ROI、目标跟踪和人工审核结合起来。具体而言，红绿灯 ROI 区域提取 HSV 颜色掩膜并与检测框空间交并比判断当前灯色，斑马线 ROI 通过几何关系判定行人或车辆是否在禁止时段越线，目标跟踪则维持同一对象的身份 ID 以消除单帧误检，使系统输出的是带持续行为特征的疑似事件，而不是孤立的检测框。这样的设计虽然增加了系统复杂度，但更符合实际应用：AI 负责发现疑似事件，人负责最终确认。报告功能的设计也体现了稳定性优先的原则，快速结构化报告用于现场演示和可靠输出，它按固定模板填入时间、地点、违法类型和抓拍证据路径，不依赖外部服务；AI 润色报告用于增强可读性，在不改变事实的前提下对语句进行流畅度优化和标准用语对齐，使处罚文书更正式。通过本次设计，我对嵌入式视觉系统从硬件连接、数据传输、模型推理到人机交互的完整流程有了更深入的理解，也认识到工程系统最重要的不只是算法本身，还包括异常处理、部署便利性和用户可操作性。比如，硬件连接需要为每一根排线预留防松脱检查，数据传输加入断线重连和序号去重，模型推理添加输入尺寸保护，前端交互设计操作指引和错误提示，这些工程细节才是系统真正可用的基石。

第五部分 参考文献

- [1] OpenCV Documentation. Open Source Computer Vision Library Documentation[EB/OL]. <https://docs.opencv.org/>.
- [2] Flask Documentation. Flask Web Development Framework[EB/OL]. <https://flask.palletsprojects.com/>.
- [3] Ultralytics. YOLO Model Documentation[EB/OL]. <https://docs.ultralytics.com/>.
- [4] Kendryte. K230 AI SoC Developer Documentation[EB/OL].
- [5] Rockchip. RK3588 Technical Reference and Developer Resources[EB/OL].
- [6] Python Software Foundation. Python Standard Library Documentation[EB/OL]. <https://docs.python.org/>.
- [7] SQLite Consortium. SQLite Documentation[EB/OL]. <https://www.sqlite.org/docs.html>.
- [8] MicroPython Documentation. MicroPython Language and Library Reference[EB/OL]. <https://docs.micropython.org/>.
- [9] 飞凌嵌入式官网及教程: <https://www.forlinux.com/> .<https://www.elfboard.com/information>